

Key Recovery on the PRESENT Block Cipher via SAT Solvers

Wu, Pin-Yu
NTUME
B11502041

I. INTRODUCTION

A. Background and Motivation

The proliferation of the Internet of Things (IoT) has led to a massive deployment of resource-constrained devices, such as RFID tags, wireless sensors, and embedded systems. These devices often have strict limitations on computational power, memory, and energy consumption, making it impractical to implement traditional cryptographic standards like the Advanced Encryption Standard (AES). This challenge has spurred the development of lightweight cryptography (LWC), a specialized field focused on creating security solutions tailored for these constrained environments.

PRESENT, proposed by Bogdanov et al. in 2007, is a prime example of an ultra-lightweight block cipher. Designed for extreme efficiency in hardware, PRESENT features a 64-bit block size and an 80-bit master key [1]. Its elegant Substitution-Permutation Network (SPN) structure and compact design led to its standardization as ISO/IEC 29192-2. As PRESENT gains widespread adoption, it is crucial to thoroughly evaluate its security against various cryptanalytic techniques.

B. Cryptanalysis and SAT Solvers

Cryptanalysis is the study of methods for obtaining the meaning of encrypted information, without access to the secret key. While traditional methods include differential and linear cryptanalysis, algebraic attacks based on constraint satisfaction have emerged as a powerful alternative.

The Boolean Satisfiability (SAT) problem is a fundamental problem in computer science, which asks whether there exists an assignment of truth values to variables that makes a given Boolean formula (in Conjunctive Normal Form, CNF) true. Although SAT is NP-complete, the performance of modern SAT solvers has improved dramatically, enabling them to solve instances with millions of variables and clauses.

This progress has made SAT-based cryptanalysis a viable attack vector. The core idea is to model the entire cryptographic algorithm—including all its logical operations and the key schedule—as a large CNF formula. A key recovery attack is then reduced to finding a satisfying assignment for this formula, given the constraints of a known plaintext-ciphertext pair. The solution found by the SAT solver will contain the secret key.

C. Objectives and Contributions

The primary objective of this paper is to investigate the feasibility and performance of a SAT-based key recovery attack on the PRESENT block cipher. We focus on creating a precise SAT model of the PRESENT-80 algorithm and evaluating the practical performance of SAT solvers in breaking reduced-round versions of the cipher.

The main contributions of this work are as follows: We provide a detailed methodology for converting each component of the PRESENT cipher—the S-box, the P-layer permutation, and the AddRoundKey (XOR) operation—into an equivalent set of CNF clauses.

We define two distinct attack models: (1) a full attack model that includes the complete 80-bit key schedule, and (2) a simplified model that assumes independent 64-bit round keys for each round.

We experimentally evaluate the performance of a SAT solver (e.g., CryptoMiniSat) in solving these two models, analyzing the time complexity and identifying the primary bottlenecks as the number of rounds increases.

Through this analysis, we aim to quantify the security margin of PRESENT against SAT-based attacks and understand the critical role of the key schedule in the cipher's overall security.

D. Paper Organization

The remainder of this paper is organized as follows. Chapter 2 introduces the necessary preliminaries, including a detailed description of the PRESENT block cipher and the fundamentals of the SAT problem. Chapter 3 elaborates on the methodology used to model PRESENT as a SAT problem. Chapter 4 details the experimental setup and formulates the two attack problems. Chapter 5 presents and analyzes the experimental results. Finally, Chapter 6 concludes the paper and suggests directions for future work.

II. PRELIMINARIES

A. The PRESENT Block Cipher

PRESENT is an ultra-lightweight block cipher designed in 2007, notable for its hardware efficiency. It operates on a 64-bit block of data ($P_i^{(j)} \in \{0, 1\}^{64}$) and uses an 80-bit master key (referred to as PRESENT-80). The encryption process consists of 31 full rounds.

1) *Overall Structure*: The 64-bit internal state is updated in each round by applying three sequential operations: sBoxLayer, pLayer, and addRoundKey.

addRoundKey: A 64-bit round key K_i (derived from the 80-bit master key) is applied to the state using a bitwise XOR operation.

sBoxLayer: A 4-bit S-box is applied 16 times in parallel to the 64-bit state.

pLayer: The 64 bits of the state are permuted.

After 31 rounds, a final addRoundKey operation is performed with a 32nd round key (K_{32}) to produce the 64-bit ciphertext.

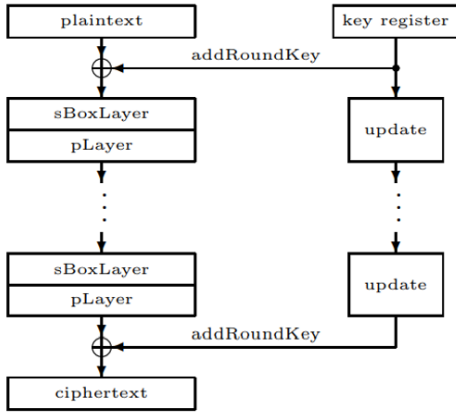


Fig. 1. A top-level algorithmic description of PRESENT

2) *The sBoxLayer*: The sBoxLayer provides confusion by applying a non-linear substitution. The 64-bit state is treated as 16 parallel 4-bit nibbles. Each nibble $X = (x_3, x_2, x_1, x_0)$ is replaced by a new 4-bit nibble $Y = (y_3, y_2, y_1, y_0)$ according to a fixed 4x4 S-box. The substitution is defined by the following lookup table:

TABLE I
THE SBOXLAYER LOOK-UP TABLE

X	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Y	C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

For example, the input nibble 0x0 (0000) is mapped to 0xC (1100), and 0x1 (0001) is mapped to 0x5 (0101).

3) *The pLayer*: The pLayer provides diffusion by permuting the bits of the state. In this permutation, bit i of the state is moved to bit position $P(i)$. The permutation rule is defined as:

$$P(i) = (16 \times i) \pmod{63} \text{ for } i \in \{0, 1, \dots, 62\}$$

$$P(63) = 63$$

This means that every bit is moved to a new position (e.g., $P(0) = 0, P(1) = 16$), except for the last bit (bit 63), which remains unchanged.

4) *The Key Schedule*: The key schedule algorithm generates 32 different 64-bit round keys ($K_1, \dots, K_{32}$) from the 80-bit

master key. The 80-bit key is stored in a key register. For each round $i = 1, \dots, 31$:

The 64 leftmost bits of the key register are extracted as the round key K_i .

The key register is rotated left by 61 bits.

The 4 leftmost bits are passed through the PRESENT S-box.

The 5-bit round counter i is XORed with bits $k_{19} \dots k_{15}$ of the register.

This process ensures that each round key is a complex, non-linear function of the original 80-bit master key, which is a critical feature for the cipher's security.

B. The Boolean Satisfiability (SAT) Problem

The Boolean Satisfiability (SAT) problem is a fundamental decision problem in computational complexity theory. The problem is to determine if there exists a truth assignment (a set of true/false values) for the variables of a given Boolean formula that makes the entire formula evaluate to true.

A Boolean formula is typically expressed in Conjunctive Normal Form (CNF). A CNF formula is a conjunction (an AND, $\wedge$) of one or more clauses, where each clause is a disjunction (an OR, $\vee$) of literals. A literal is either a Boolean variable (x) or its negation ($\neg x$).

For example, the following is a CNF formula:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_3)$$

A SAT solver is an algorithm that takes a CNF formula as input and outputs one of two results:

SATISFIABLE (SAT): If a satisfying assignment exists, the solver will return it (e.g., for the formula above, the assignment $x_1=\text{false}, x_2=\text{false}, x_3=\text{false}$ is a solution).

UNSATISFIABLE (UNSAT): If no such assignment exists.

In the context of cryptanalysis, the variables represent the unknown key bits and intermediate state bits, and the clauses represent the logical constraints imposed by the cipher's operations.

III. CNF MODELING OF PRESENT

To analyze a block cipher using a SAT solver, the entire cryptographic process must be translated into a Boolean formula in Conjunctive Normal Form (CNF). This involves representing all internal state bits, key bits, and intermediate values as Boolean variables, and representing all cipher operations as a set of clauses that constrain these variables.

A. Modeling Approach

Our model converts the r -round encryption path of PRESENT, from plaintext P to ciphertext C , into a SAT instance. For each round j (from 0 to $r-1$), we define Boolean variables for the state:

$P_i^{(j)}$: The i -th bit of the state before addRoundKey in round j . (Where $P^{(0)}$ is the plaintext).

$S_i^{(j)}$: The i -th bit of the state after the sBoxLayer in round j .

$Z_i^{(j)}$: The i -th bit of the state after the pLayer in round j .

$K_i^{(j)}$: The i -th bit of the round key K_j .

The state $P^{(j+1)}$ (the input to the next round) is the same as $Z_i^{(j)} \oplus K_i^{(j)}$. The final ciphertext C is the state after the final `addRoundKey` operation of the r -th round.

We now describe the CNF transformation for each of the cipher's core operations.

B. CNF Transformation of the *sBoxLayer*

The *sBoxLayer* applies 16 independent 4x4 S-boxes. Each S-box maps a 4-bit input $X = (x_3, x_2, x_1, x_0)$ to a 4-bit output $Y = (y_3, y_2, y_1, y_0)$. We can encode the S-box's lookup table directly into CNF.

A common method is to encode all 16 entries of the table. For each entry (e.g., $Input \rightarrow Output$), we create a set of clauses. An input X (e.g., $X = 0$, which is 0000_2) is represented as a conjunction of literals:

$$(\neg x_3 \wedge \neg x_2 \wedge \neg x_1 \wedge \neg x_0)$$

The corresponding output Y (e.g., $Y = C$, which is 1100_2) is represented by four implications:

$$\begin{aligned} (X = 0) &\rightarrow y_3 \\ (X = 0) &\rightarrow y_2 \\ (X = 0) &\rightarrow \neg y_1 \\ (X = 0) &\rightarrow \neg y_0 \end{aligned}$$

Using the Boolean identity $(A \rightarrow B) \equiv (\neg A \vee B)$ and De Morgan's laws, the first implication $(\neg x_3 \wedge \neg x_2 \wedge \neg x_1 \wedge \neg x_0) \rightarrow y_3$ is transformed into a single clause:

$$(x_3 \vee x_2 \vee x_1 \vee x_0 \vee y_3)$$

Since there are 4 output bits for each input, we get 4 clauses for each of the 16 entries in the S-box table.

$$4 \text{ clauses/entry} \times 16 \text{ entries} = 64 \text{ clauses per round}$$

This set of 64 clauses fully describes the logical behavior of a single 4x4 S-box.

C. CNF Transformation of the *pLayer*

The *pLayer* is a simple bit permutation, $Z_{P(i)}^{(j)} \leftrightarrow S_i^{(j)}$. This is a logical equivalence: $A \leftrightarrow B$. This can be expressed in CNF with two clauses:

$$(A \rightarrow B) \wedge (B \rightarrow A) \equiv (\neg A \vee B) \wedge (\neg B \vee A)$$

For each of the 64 bits in the state, we introduce 2 clauses to model the permutation. Therefore, the *pLayer* adds:

$$64 \text{ bits} \times 2 \text{ clauses/bit} = 128 \text{ clauses per round}$$

D. CNF Transformation of *AddRoundKey(XOR)*

The `addRoundKey` operation is a bitwise XOR: $P_i^{(j+1)} = Z_i^{(j)} \oplus K_i^{(j)}$. A 3-variable XOR operation $A = B \oplus C$ is equivalent to the following four clauses, which cover all possible truth assignments:

$$\begin{aligned} (\neg A \vee \neg B \vee C) \\ (\neg A \vee B \vee \neg C) \\ (A \vee \neg B \vee \neg C) \\ (A \vee B \vee C) \end{aligned}$$

This operation is applied to all 64 bits of the state. Therefore, the `addRoundKey` layer adds:

$$64 \text{ bits} \times 4 \text{ clauses/bit} = 256 \text{ clauses per round}$$

Based on these calculations, the CNF model for a single encryption round (excluding the key schedule) requires $1024 + 128 + 256 = 1408$ clauses. As this cost is fixed per round, the total number of clauses for an r -round model scales linearly.

E. Instance Scale with Key Schedule

When the PRESENT-80 key schedule is included, measured CNF sizes on our generator grow approximately linearly with the round count r :

$$\text{vars} \approx 724 + 436(r - 1), \quad \text{clauses} \approx 2496 + 1952(r - 1). \quad (1)$$

In particular, the full 31-round instance has 13 804 variables and 61 056 clauses (CNF file size about 1.4 MB). The instance size itself is modest for modern SAT solvers; the practical difficulty arises from search structure after cryptographic diffusion rather than from raw file size.

IV. EXPERIMENTAL SETUP

A. Threat Model

We consider a known-plaintext setting with a single 64-bit plaintext/ciphertext pair and a truncated round count r . The attacker does not know the 80-bit master key and aims to recover any master key consistent with the key schedule such that $\text{Enc}_r(PT, K) = CT$. Side-channel attacks, fault injection, and multi-pair distinguishing attacks are out of scope. The study is educational and validation-oriented: it quantifies how SAT difficulty grows with r , and does not claim a practical break of full 31-round PRESENT-80.

B. Solver Implementation

The attack was implemented using the **CryptoMiniSat** SAT solver, which features native support for XOR clause extensions. The plaintext-ciphertext pairs used for the attack were randomly generated 64-bit binary numbers.

Supplemental timed benchmarks used **CryptoMiniSat 5.13.0** on Linux (WSL2), Intel Core i7-13620H, with a wall-clock limit of `--maxtime=30` seconds per instance. Recovered keys were verified by re-encrypting with a reference implementation aligned to the CNF semantics.

C. Problem Formulations

We evaluated two distinct attack scenarios to understand the complexity of the cipher:

- 1) **Full Attack Model:** The goal is to recover the 80-bit master key given plaintext-ciphertext pairs. This model includes constraints for both the encryption path and the complex key schedule.
- 2) **Simplified Attack Model:** In this scenario, we treat the r round keys as independent 64-bit values, ignoring the key schedule rules.

V. RESULTS AND ANALYSIS

A. Full Attack Results

Experimental results for two test cases indicate that recovering the master key becomes exponentially more difficult as the number of rounds increases.

TABLE II
RESULT (TESTCASE 1)

Round	Variables	Clauses	SAT Solver Time
1	724	2496	instantly
2	1160	4448	instantly
3	1596	6400	85 s
4	2032	8352	55 min
5	2468	10304	> 3 hr

In Testcase 2, the solver failed to find a solution within 4 hours after only 4 rounds.

TABLE III
RESULT (TESTCASE 2)

Round	Variables	Clauses	SAT Solver Time
3	1596	6400	13 min
4	2032	8352	> 4 hr

B. Fixed-Time Budget and Full-Round Attempt

Under a fixed 30-second solver budget on the same full-attack encoding (testcase 1), rounds 1–2 remain SAT, while rounds 3–5 and the full 31-round instance all time out. Round 31 has 13 804 variables and 61 056 clauses yet yields no decisive answer within 30 s. These short-budget timeouts are consistent with Table II: satisfying assignments for small r exist and were previously found under longer runtimes, but a short wall-clock limit is insufficient once diffusion accumulates.

TABLE IV
FIXED 30 S BUDGET (TESTCASE 1, CRYPTOMINISAT 5.13.0)

Round	Variables	Clauses	Result	Wall time (s)
1	724	2496	SAT	0.004
2	1160	4448	SAT	0.115
3	1596	6400	TIMEOUT	29.204
4	2032	8352	TIMEOUT	29.311
5	2468	10304	TIMEOUT	29.201
31	13804	61056	TIMEOUT	29.210

Thus, full 31-round key recovery via plain SAT is not expected to finish in practice on commodity hardware within

acceptable interactive time (minutes to a few hours). The CNF remains small; the bottleneck is CDCL search after stacked nonlinear layers, compounded by a single known-plaintext pair (64-bit ciphertext vs. 80-bit key), so solutions need not be unique.

C. Simplified Attack Results

When the key schedule constraints are removed, the SAT solver can handle a significantly higher number of rounds.

TABLE V
RESULT WITH SIMPLIFIED METHOD

Round	Variables	Clauses	SAT Solver Time
10	2752	14464	instantly
100	25792	141184	instantly
1000	256192	1408384	42 s
5000	1280192	7040384	65 s
10000	2560192	14080384	killed

The contrast between the full and simplified models demonstrates that the primary bottleneck for SAT-based key recovery in PRESENT is the non-linear key schedule rather than the encryption path itself.

VI. CONCLUSION

This study quantifies the security margin of the PRESENT block cipher against SAT-based algebraic attacks. Our findings highlight that while the encryption logic is susceptible to SAT modeling, the 80-bit master key schedule provides strong protection, effectively preventing recovery beyond 4–5 rounds within reasonable timeframes. Complementary fixed-budget runs further show that the full 31-round PRESENT-80 instance times out quickly despite a modest CNF size, so plain SAT master-key recovery at full round count is not expected to complete in practice.

REFERENCES

- [1] A. Bogdanov et al., “PRESENT: An Ultra-Lightweight Block Cipher.